

# RETRIEVAL-GENERATION SYNERGY AUGMENTED LARGE LANGUAGE MODELS

Zhangyin Feng, Xiaocheng Feng, Dezhi Zhao, Maojin Yang, Bing Qin

Harbin Institute of Technology, China

## ABSTRACT

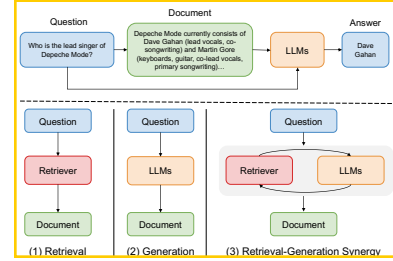
Large language models augmented with task-relevant documents have demonstrated impressive performance on knowledge-intensive tasks. However, regarding how to obtain effective documents, the existing methods are mainly divided into two categories. One is to retrieve from an external knowledge base, and the other is to utilize large language models to generate documents. We propose an iterative retrieval-generation collaborative framework. It is not only able to leverage both parametric and non-parametric knowledge, but also helps to find the correct reasoning path through retrieval-generation interactions, which is very important for tasks that require multi-step reasoning. We conduct experiments on four question answering datasets, including single-hop QA and multi-hop QA tasks. Empirical results show that our method significantly improves the reasoning ability of large language models and outperforms previous baselines.

**Index Terms**— large language models, retrieval augmented, question answering

## 1. INTRODUCTION

Large Language models (LLMs) have demonstrated impressive performance on diverse language tasks through in-context learning [1, 2, 3, 4, 5, 6]. However, they still struggle with knowledge-intensive tasks that require access to a large amount of knowledge, such as open-domain question answering [7] and commonsense reasoning [8], since the implicit knowledge preserved in the parameters may be partial and insufficient. As shown in the top of Figure 1, one promising direction is to incorporate non-parametric knowledge to help alleviate this problem with large language models.

Recent research shows that retrieving relevant documents from an external datastore [9, 10, 11] or directly generating contextual documents from LLMs [12, 13] both can improve LLMs’ performance on knowledge-intensive tasks. The former, called retrieve-then-read, requires a retriever to retrieve relevant documents. The latter, known as generate-then-read, leverages large language models to generate relevant documents before answering questions. However, as shown in Figure 1, the above two methods are isolated and lack coordination with each other. To fill this gap, in this paper, we explore an effective retrieval-generation collaboration frame-



**Fig. 1:** The top is the standard method utilizing LLMs for question answering with relevant documents. The bottom shows three methods to generate relevant documents.

work to further improve the ability of large language models to solve knowledge-intensive tasks.

In this work, we present ITRG, an **IT**erative **R**etrieval-**G**eneration synergy framework to generate relevant documents that simultaneously exploits parametric and non-parametric knowledge. In each iteration, ITRG consists of two important steps: **generation augmented retrieval (GAR)** and **retrieval augmented generation (RAG)**. In the GAR step, we propose a simple and effective method to expand queries by concatenating pseudo-documents generated from large language models and original questions. And expanded queries improve the accuracy of retrieving relevant documents. In the RAG step, we use large language models to comprehensively understand retrieved documents to generate new documents for answering questions. We repeat these steps until we reach the maximum allowed number of iterations. Through multiple retrieval generation collaborations, our method aids in discovering the appropriate reasoning path and providing correct answers to questions.

We evaluate the efficacy of our method on 4 question answering datasets, including Natural Questions, TriviaQA, 2WikiMultiHopQA, and HotpotQA. Experimental results show that our method performs better than previous baselines on all datasets. In summary, our main contributions can be summarized as follows: (1) We propose ITRG, an iterative retrieval-generation synergy framework using both parametric and non-parametric knowledge. (2) We propose a simple and effective generation-augmented retrieval strategy and two retrieval-augmented generation strategies. (3) Empirical results show that ITRG outperforms previous retrieval-augmented methods.

## 2. ITERATIVE RETRIEVAL-GENERATION SYNERGY

In this section, we first introduce the overall framework, and then introduce the retrieval-generation collaboration framework in detail, including generation augmented retrieval and retrieval augmented generation.

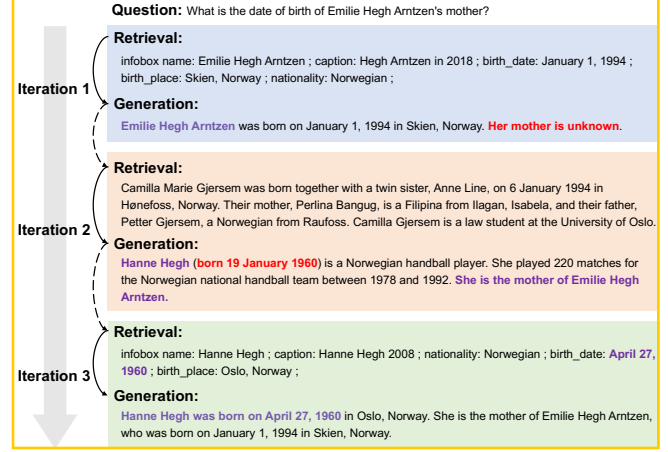
### 2.1. Overview

We show the framework of ITRG in Figure 2. Given a user question  $q$  and a document corpus  $\mathcal{D} = \{d_i\}_{i=1}^{|\mathcal{D}|}$  (i.e.,  $d_i$  is a Wikipedia paragraph.), ITRG repeats generation augmented retrieval (GAR) and retrieval augmented generation (RAG) for  $T$  iterations. In the GAR process of iteration  $t$ , we concatenate the output  $y_{t-1}$  of the last iteration and question  $q$  to form a new query, and then use a dense retriever to retrieve top- $k$  paragraphs. In the first iteration, we only use the question as the query. In the RAG process of iteration  $t$ , based on the question  $q$  and the retrieved top- $k$  paragraphs, we exploit large language models to generate new paragraphs to answer questions. Specifically, we propose two methods to generate new paragraphs, which will be introduced in detail in §2.3.

### 2.2. Generation Augmented Retrieval

Knowledge-intensive tasks (e.g., open-domain question answering) often require access to additional documents. A common approach is to directly employ the question as the query, and then equip a sparse or dense retriever to retrieve relevant documents. In practice, we find that in some cases using the question directly as the query fails to retrieve relevant documents because there may exist semantic gaps between them. To alleviate this problem, we propose a simple query expansion method. At the first iteration  $t = 1$ , we use the original question  $q$  as the query. At iteration  $t$  ( $t > 1$ ), we concatenate the original question  $q$  and the document generated  $y_{t-1}$  in the last iteration as the new query  $q_t = [q; y_{t-1}]$ . Then, we utilize a pre-trained dense retriever to retrieve top- $k$  documents, which are denoted as  $R_t = \{d\}$ .

Given an input question  $q$ , the retriever aims to retrieve a small set of documents from a corpus  $\mathcal{D} = \{d_i\}_{i=1}^{|\mathcal{D}|}$  that are relevant to  $q$ . Following prior work [14], we use a dense retriever based on the dual encoder architecture, where an encoder is used to encode both the input context  $q$  and the document  $d$ . Specifically, the encoder maps each document  $d \in \mathcal{D}$  to an embedding  $\mathbf{E}(d)$  by taking the mean pooling of the last hidden representation over the tokens in  $d$ . At query time, the same encoder is applied to the input context  $q$  to obtain a query embedding  $\mathbf{E}(q)$ . The similarity between the query embedding and the document embedding is computed by their cosine similarity:  $s(d, q) = \cos(\mathbf{E}(d), \mathbf{E}(q))$ . The top- $k$  documents that have the highest similarity scores are retrieved.



**Fig. 2:** Iterative retrieval-generation synergy framework contains two steps in each iteration: (1) generation augmented retrieval (GAR): utilize the output of the previous iteration to expand the query to help retrieve more relevant documents; (2) retrieval augmented generation (RAG): utilize retrieved documents to generate new documents to answer questions. We only show three iterations in this figure for brevity. Solid arrows indicate RAG within an iteration, and dashed arrows indicate GAR between iterations. Purple represents correct and useful information, and red represents wrong or invalid information.

### 2.3. Retrieval Augmented Generation

Following previous work [13], for a given question  $q$ , we could directly prompt large language models to generate related documents without retrieving them from an external corpus. However, we find that if only the parametric knowledge learned by the large model in the pre-training stage is used, the generated documents may be incomplete. Retrieval augmented generation (RAG) aims to comprehensively understand the retrieved non-parametric knowledge and the parametric knowledge inside large language models to generate more accurate factual knowledge. Specifically, we propose two strategies, which will be described in detail below.

#### 2.3.1. Refine

An intuitive idea is to refine the previously generated document  $y_{t-1}$  based on the original question  $q$  and the retrieved top- $k$  documents at the current iteration step  $R_t$  to obtain a new document  $y_t$ . We call this method refine. Considering that the document retrieved in the last iteration  $R_{t-1}$  has been used to generate the last document  $y_{t-1}$ , we refine the previous output  $y_{t-1}$  with updated documents  $R_{update}$ .

$$R_{update} = R_t - R_{t-1}, \quad (1)$$

$$y_t = \mathcal{M}(\text{prompt}(y_{t-1}, q, R_{update})), \quad (2)$$

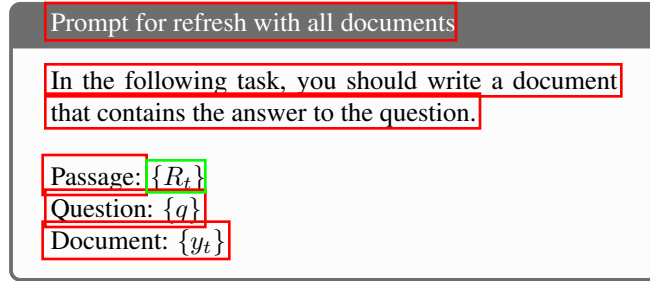
where  $R_{update}$  means that these documents are only retrieved in the current iteration, not in the last iteration,  $\mathcal{M}$  denotes a well pre-trained large language model. If  $R_{update}$  is an empty set, we do not regenerate a new document and set  $y_t = y_{t-1}$ .

### 2.3.2. Refresh

In order to avoid the negative effect of errors or hallucinations in the previously generated document  $y_{t-1}$ , we do not use  $y_{t-1}$ , which is used in refine. We refresh the memory and let the large language models directly generate the document  $y_t$  based on the retrieved document  $R_t$  and the original question  $q$ . This method is named refresh.

$$y_t = \mathcal{M}(\text{prompt}(q, R_t)) \quad (3)$$

Both refine and refresh are implemented through prompts. We give the prompt corresponding to refresh.



## 3. EXPERIMENTAL SETUP

### 3.1. Datasets

We evaluate the effectiveness of ITRG on four open domain question answering datasets, including Natural Questions (NQ) [15], TriviaQA [16], 2WikiMultiHopQA [17] and HotpotQA [18]. Following previous works [19, 20], we randomly sub-sample 500 examples from each dataset due to the cost of running experiments. We evaluate our method in 0-shot, 1-shot and 5-shot settings. The few-shot demonstrations are randomly sampled from the data that is not involved in the evaluation process.

### 3.2. Baselines

**GPT-3.5** [21] We use text-davinci-002 and text-davinci-003 as our baselines. Text-davinci-002 is an InstructGPT model while Text-davinci-003 is trained with reinforcement learning with reward models trained from comparisons by humans. **Vanilla LM** The vanilla LM baselines prompt an LLM to directly generate an answer following the few-shot in-context learning paradigm [1]. **CoT** We follow [22] to generate both the chain-of-thought (CoT) reasoning process and the final answer. We only evaluate this method on multi-hop reasoning datasets in 5-shot setting<sup>1</sup>. **Retrieve-then-Read** The retrieve-

<sup>1</sup>We also conduct evaluation in 1-shot setting, but the final answer could not be generated according to the corresponding instructions

then-read baseline consists of a well-pre-trained dense retriever and a large language model. The retriever retrieves relevant documents for the question, and then the LLM conditions on both the question and retrieved documents to generate the answer. **Generate-then-Read** Generate-then-read baseline first uses few-shot prompts to generate a question-related document, and then concatenates it with the question to regenerate the answer.

### 3.3. Details

LLaMA [6] is an open source well trained large language model. Considering the performance and computational cost of the model, we use LLaMA 33B as the backend LLM. We use greedy decoding for both document generation and answer generation, and set up to generate 200 tokens and 15 tokens respectively. We retrieve the top-5 paragraphs for each query and set the maximum number of iterations  $T$  to 5. We directly use the pre-trained dense retriever [23] and used the December 2018 Wikipedia dump as the retrieval corpus for all datasets. Generated answers are evaluated with the standard exact match metric (EM score): a generated answer is considered correct if it matches any answer of the list of answers after normalization. For this normalization step, we lower-case generated answers and remove articles, punctuation and duplicate whitespaces.

## 4. RESULTS

### 4.1. Main Results

Table 1 reports the results on the single-hop question answering datasets. In the 1-shot and 5-shot settings, the performance of LLaMA-33B based Vanilla LM is very close to that of text-davinci-003. This shows LLaMA-33B is a strong language model, and it is reasonable to choose LLaMA-33B as our backend LLM. Retrieve-then-read and generate-then-read all exceed vanilla LM, verifying that adding relevant external knowledge can improve the reasoning ability of large language models. In addition, we observe that our iterative retrieval-generation collaborative method ITRG achieves state-of-the-art performance on both datasets. Specifically, ITRG (refresh) performs better on the NQ dataset, and ITRG (refine) performs better on the TriviaQA dataset.

Table 2 presents the results on the multi-hop question answering datasets. We observe that LLaMA-33B is still comparable to text-davinci-003 on the multi-hop question answering datasets. In addition, CoT can answer questions more accurately than vanilla LM by generating reasoning process. Compared with different baseline models, ITRG significantly improves the exact match scores. Specifically, on the 2Wiki-MultiHopQA dataset, the exact match score of ITRG (refresh) in the zero-shot setting is 32.2, which exceeds the performance of vanilla LM in the 5-shot setting with a score of 31.8. In the 5-shot setting, ITRG (refresh) achieves 38.6 EM score

**Table 1:** Exact match performance on single-hop question answering. All ITRG results are from the last iteration ( $T = 5$ ).

| Method    |                    | Natural Questions |             |             | TriviaQA    |             |             |
|-----------|--------------------|-------------------|-------------|-------------|-------------|-------------|-------------|
|           |                    | 0-shot            | 1-shot      | 5-shot      | 0-shot      | 1-shot      | 5-shot      |
| GPT 3.5   | Text-davinci-002   | 12.0              | 24.6        | 33.0        | 46.0        | 74.2        | 76.0        |
|           | Text-davinci-003   | 29.4              | 33.0        | 33.8        | 75.8        | 78.6        | 77.8        |
| LLaMA 33B | Vanilla LM         | 27.0              | 29.4        | 32.4        | 74.8        | 70.8        | 75.8        |
|           | Retrieve-then-Read | 27.8              | 30.6        | 29.8        | 74.6        | 76.0        | 76.0        |
|           | Generate-then-Read | 28.0              | 31.4        | 31.0        | 73.6        | 77.2        | 77.6        |
|           | ITRG (refine)      | 34.4              | 34.6        | 34.8        | <b>79.0</b> | <b>79.4</b> | <b>80.6</b> |
|           | ITRG (refresh)     | <b>37.6</b>       | <b>38.4</b> | <b>38.0</b> | 77.0        | 78.6        | 79.4        |

**Table 2:** Exact match performance on multi-hop question answering. All ITRG results are from the last iteration ( $T = 5$ ).

| Method    |                    | 2WikiMultiHopQA |             |             | HotpotQA    |             |             |
|-----------|--------------------|-----------------|-------------|-------------|-------------|-------------|-------------|
|           |                    | 0-shot          | 1-shot      | 5-shot      | 0-shot      | 1-shot      | 5-shot      |
| GPT 3.5   | Text-davinci-002   | 16.4            | 27.6        | 30.8        | 12.2        | 20.2        | 22.2        |
|           | Text-davinci-003   | 27.2            | 27.0        | 29.8        | 25.0        | 25.8        | 26.6        |
| LLaMA 33B | Vanilla LM         | 24.4            | 27.6        | 31.8        | 22.6        | 25.0        | 27.0        |
|           | COT                | -               | -           | 32.2        | -           | -           | 28.6        |
|           | Retrieve-then-Read | 27.4            | 29.2        | 32.0        | 28.4        | 29.8        | 30.4        |
|           | Generate-then-Read | 30.0            | 30.4        | 31.6        | 25.0        | 27.0        | 27.0        |
|           | ITRG (refine)      | <b>33.0</b>     | 33.6        | 37.0        | 28.8        | 29.6        | 30.6        |
|           | ITRG (refresh)     | 32.2            | <b>36.2</b> | <b>38.6</b> | <b>31.0</b> | <b>32.6</b> | <b>33.4</b> |

**Table 3:** Exact match performance of ITRG (refresh) at different iterations in 5-shot setting.

| Iteration         | 1    | 2    | 3    | 4    | 5    |
|-------------------|------|------|------|------|------|
| Natural Questions | 34.0 | 35.2 | 37.0 | 37.2 | 38.0 |
| TriviaQA          | 79.8 | 79.2 | 79.8 | 79.8 | 79.4 |
| 2WikiMultiHopQA   | 34.8 | 37.4 | 37.2 | 38.6 | 38.6 |
| HotpotQA          | 32.6 | 32.8 | 34.0 | 33.4 | 33.4 |

**Table 4:** Answer recall of generated documents at different iterations with ITRG (refresh).

| Iteration         | 1    | 2    | 3    | 4    | 5    |
|-------------------|------|------|------|------|------|
| Natural Questions | 44.0 | 46.4 | 48.4 | 48.8 | 48.0 |
| TriviaQA          | 18.8 | 19.0 | 20.2 | 19.2 | 19.2 |
| 2WikiMultiHopQA   | 34.2 | 36.6 | 35.0 | 40.0 | 37.0 |
| HotpotQA          | 34.2 | 34.8 | 35.6 | 33.8 | 33.6 |

and improves by 6.8 points in absolute gains. Compared to vanilla LM, ITRG (refresh) can improve the EM score by 9.4, 7.6, and 6.4 points respectively in 0-shot, 1-shot, and 5-shot settings on the Hotpotqa dataset.

framework is effective and can further enhance the reasoning capabilities of large language models.

## 5. CONCLUSION

### 4.2. Performance at Different Iterations

In this section, we analyze the performance of our model and the quality of the generated documents during the iteration process. Specifically, we present the results of ITRG (refresh) at different iterations in 5-shot setting in Table 3. We measure the answer recall of generated documents at different iteration steps and present results in Table 4. Table 3 shows that the performance of the model gradually improves with iteration. And Table 4 shows that the quality of the generated documents also gradually improves with iteration. These results verify that our iterative retrieval-generation collaborative

In this paper, we present ITRG, which is an iterative retrieval-generation synergy framework, containing two important steps: generation-augmented retrieval and retrieval-augmented generation. They form a closed loop, and can improve each other via multiple iterations. We propose a simple and effective generation-augmented retrieval strategy and two retrieval-augmented generation strategies. Empirical results show our approach significantly exceeds several strong baselines, including GPT 3.5, on four open domain question answering datasets, which indicates that our method can significantly improve the reasoning ability of large language models.

## 6. REFERENCES

- [1] T. Brown *et al.*, “Language models are few-shot learners,” *Advances in neural information processing systems*, vol. 33, pp. 1877–1901, 2020.
- [2] J. Hoffmann *et al.*, “Training compute-optimal large language models,” 2022.
- [3] A. Zeng *et al.*, “Glm-130b: An open bilingual pre-trained model,” *arXiv preprint arXiv:2210.02414*, 2022.
- [4] A. Chowdhery *et al.*, “Palm: Scaling language modeling with pathways,” *arXiv preprint arXiv:2204.02311*, 2022.
- [5] OpenAI, “Gpt-4 technical report,” 2023.
- [6] H. Touvron *et al.*, “Llama: Open and efficient foundation language models,” 2023.
- [7] K. Lee, M.-W. Chang, and K. Toutanova, “Latent retrieval for weakly supervised open domain question answering,” in *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*. Florence, Italy: Association for Computational Linguistics, Jul. 2019, pp. 6086–6096. [Online]. Available: <https://aclanthology.org/P19-1612>
- [8] R. Zellers, Y. Bisk, R. Schwartz, and Y. Choi, “SWAG: A large-scale adversarial dataset for grounded common-sense inference,” in *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*. Brussels, Belgium: Association for Computational Linguistics, Oct.-Nov. 2018, pp. 93–104. [Online]. Available: <https://www.aclweb.org/anthology/D18-1009>
- [9] O. Ram *et al.*, “In-context retrieval-augmented language models,” *arXiv preprint arXiv:2302.00083*, 2023.
- [10] O. Khattab *et al.*, “Demonstrate-search-predict: Composing retrieval and language models for knowledge-intensive nlp,” 2023.
- [11] W. Shi *et al.*, “Replug: Retrieval-augmented black-box language models,” *arXiv preprint arXiv:2301.12652*, 2023.
- [12] W. Yu *et al.*, “Generate rather than retrieve: Large language models are strong context generators,” 2023.
- [13] Z. Sun, X. Wang, Y. Tay, Y. Yang, and D. Zhou, “Recitation-augmented language models,” 2023.
- [14] G. Izacard and E. Grave, “Leveraging passage retrieval with generative models for open domain question answering,” *arXiv preprint arXiv:2007.01282*, 2020.
- [15] T. Kwiatkowski *et al.*, “Natural questions: A benchmark for question answering research,” *Transactions of the Association for Computational Linguistics*, vol. 7, pp. 452–466, 2019. [Online]. Available: <https://aclanthology.org/Q19-1026>
- [16] M. Joshi, E. Choi, D. Weld, and L. Zettlemoyer, “TriviaQA: A large scale distantly supervised challenge dataset for reading comprehension,” in *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Vancouver, Canada: Association for Computational Linguistics, Jul. 2017, pp. 1601–1611. [Online]. Available: <https://aclanthology.org/P17-1147>
- [17] X. Ho, A.-K. Duong Nguyen, S. Sugawara, and A. Aizawa, “Constructing a multi-hop QA dataset for comprehensive evaluation of reasoning steps,” in *Proceedings of the 28th International Conference on Computational Linguistics*. Barcelona, Spain (Online): International Committee on Computational Linguistics, Dec. 2020, pp. 6609–6625. [Online]. Available: <https://aclanthology.org/2020.coling-main.580>
- [18] Z. Yang *et al.*, “HotpotQA: A dataset for diverse, explainable multi-hop question answering,” in *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*. Brussels, Belgium: Association for Computational Linguistics, Oct.-Nov. 2018, pp. 2369–2380. [Online]. Available: <https://aclanthology.org/D18-1259>
- [19] H. Trivedi, N. Balasubramanian, T. Khot, and A. Sabharwal, “Interleaving retrieval with chain-of-thought reasoning for knowledge-intensive multi-step questions,” *arXiv preprint arXiv:2212.10509*, 2022.
- [20] Z. Jiang *et al.*, “Active retrieval augmented generation,” *arXiv preprint arXiv:2305.06983*, 2023.
- [21] L. Ouyang *et al.*, “Training language models to follow instructions with human feedback,” *Advances in Neural Information Processing Systems*, vol. 35, pp. 27 730–27 744, 2022.
- [22] J. Wei *et al.*, “Chain of thought prompting elicits reasoning in large language models,” *arXiv preprint arXiv:2201.11903*, 2022.
- [23] G. Izacard *et al.*, “Few-shot learning with retrieval augmented language models,” *arXiv preprint arXiv:2208.03299*, 2022.